CHILD HEALTH SYSTEM

IT 23431 –MongoDB Essentials

Submitted by

SURIYA NARAYANAN S-241001278

SUMANTH K -241001273

SRIHARI G V-241001261

OF

BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)

RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project titled "Child Health System" is the bonafide work of "***SURIYA NARAYANAN S (241001278), SUMANTH K (241001273), SRIHARI G V (241001261***)" who carried out the project work under my supervision.

SIGNATURE

Dr.P.Valarmathie

HEAD OF THE DEPARTMENT
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam
Chennai – 602105

SIGNATURE

Mrs.Usha S

COURSE IN CHARGE
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam
Chennai - 602105

This project is submitted for IT23431 – MongoDB Essentials held on _____

INTERNAL EXAMINAR ***EXTERNAL EXAMINAR***

TABLE OF CONTENTS

CHILD HEALTH SYSTEM

1.1 ABSTRACT

The Child Health System is a digital platform designed to automate the tracking of pediatric growth and health metrics. This system allows for the secure storage of child details, including height, weight, and blood group, and generates unique identification codes for every record. By involving healthcare workers, parents, and pediatricians in a single ecosystem, the project streamlines health monitoring and provides real-time alerts for children needing medical attention, such as those flagged for obesity or malnourishment.

The core objective of the Child Health System is to provide a scalable and reliable solution for public health monitoring. By utilizing MongoDB's NoSQL flexibility, the system efficiently handles dynamic health records, reduces the administrative burden on health workers, and improves the overall quality of pediatric care through timely data visualization and role-specific alerts.

1.2 INTRODUCTION

The Child Health System is a full-stack web application developed using React.js, Node.js, Express, and MongoDB. In traditional healthcare settings, tracking pediatric growth often involves manual data entry on paper cards, which are prone to damage or loss. This project automates the process, allowing healthcare workers to securely input vital metrics like height and weight and instantly generate a 6-digit unique code for each child. By providing specialized portals for Parents and Pediatricians, the system ensures that health data is not just stored, but used to provide actionable health tips and medical intervention for children flagged as "Malnourished" or "Obese".

1.3 PURPOSE

The primary purpose of this system is to maintain an organized, productive, and secure digital registry for pediatric health.

Key objectives include:

Automation: Replacing manual record-keeping with an automated digital system.

Accessibility: Providing parents with instant access to health reports via unique identification codes.

Proactive Care: Enabling pediatricians to quickly identify children needing urgent medical attention through a dedicated "Attention Required" dashboard.

1.4 SCOPE OF THE PROJECT

The scope covers the entire lifecycle of a pediatric health record:
User Management: Implementing role-based access for Workers, Parents, and Pediatricians.
Growth Tracking: Monitoring real-time metrics including Height, Weight, and Blood Group.
Health Analytics: Automatically categorizing children based on health status (e.g., Healthy, Malnourished).
Data Security: Ensuring records are only accessible to authorized users using secure login credentials.

1.5 SOFTWARE REQUIREMENTS OF THE PROJECT

Operating System: Windows 10 / Windows 11 / Linux.

Front-End Technologies: React.js, Lucide-React (for icons), and CSS.

Back-End Technologies: Node.js and Express.js.

Database: MongoDB (NoSQL) for flexible health record storage.

Code editor: Visual Studio Code (IDE)

Web browser: Google chrome / Mozilla firefox

Version Control System: Git and Github

Based on the technical stack and configuration of your Child Health System, here are the software requirements detailed in the same professional format:
The software requirements for the Child Health System include a suitable operating system such as Windows 10, Windows 11, or Linux to ensure smooth execution of the application. The system is developed using the MERN stack, with React.js and Lucide-React used for designing the modern, responsive user interface. For data storage and management, MongoDB is used as the NoSQL backend database, while Mongoose is utilized to establish secure connectivity and object modeling between the Node.js/Express server and the database.

Product Functionality

a) Role-Based Login: Secure access for Healthcare Workers, Pediatricians, and Parents to their specific portals.

b) Child Registration: Allows workers to register children with details like name, age, height, and weight.

c) Unique ID Generation: Automatically assigns a random 6-digit identification code to each child.

d) Search Feature: Enables parents and staff to find health records using the unique code or name.

e) Health Status Labels: Automatically flags children as "Healthy," "Obese," or "Malnourished" based on their metrics.

f) Consultation Updates: Allows pediatricians to view children needing attention and mark them as "Consulted".

Hardware Requirements

The minimum hardware requirements to develop and use the application are listed below:

Processor: Intel Core i3 or above
RAM: Minimum 4 GB
Hard Disk: Minimum 100 GB free space
Display: 14-inch monitor or higher resolution
Input Devices: Keyboard and Mouse
Internet Connection: Required for accessing the deployed application and real-time MongoDB Atlas connectivity

Assumptions and Dependencies

Administrators or the system seeding process will create and manage initial login credentials for authorized roles (e.g., worker1, parent1).

The system depends on MongoDB for storing all health-related data, including child profiles, user credentials, and consultation logs.

The system assumes that the Node.js/Express backend is active and reachable via the API URL defined in the environment variables.

Functional Requirements

1. Login Module (LM)

Users (Healthcare Workers, Pediatricians, and Parents) log in using their credentials. Access to specific dashboards is granted only after verification with the MongoDB database to ensure role-based security.

---

2. Child Registration Module (CRM)

Healthcare workers can add new child records including name, age, height, weight, and blood group. Upon saving, the system automatically generates a 6-digit unique identification code for the child.

---

3. Health Status Module (HSM)

The system automatically calculates the child's health category (e.g., Healthy, Obese, or Malnourished) based on the input metrics. It provides real-time status updates and tailored "Tips for Betterment".

---

4. Parent Search Module (PSM)

Allows parents to securely retrieve their child’s health report. The system searches the database via the unique code or child’s name and displays the current health status and consultation history.

---

5. Pediatrician Consultation Module (PCM)

Pediatricians can view a list of children flagged with health anomalies. They can review the metrics and update the record to mark it as "Consulted" using a secure update request to the database.

---

6. Server Module (SM)

Handles all communication between the React frontend and the MongoDB database. It validates requests, processes health status logic, and ensures data consistency across all user portals.

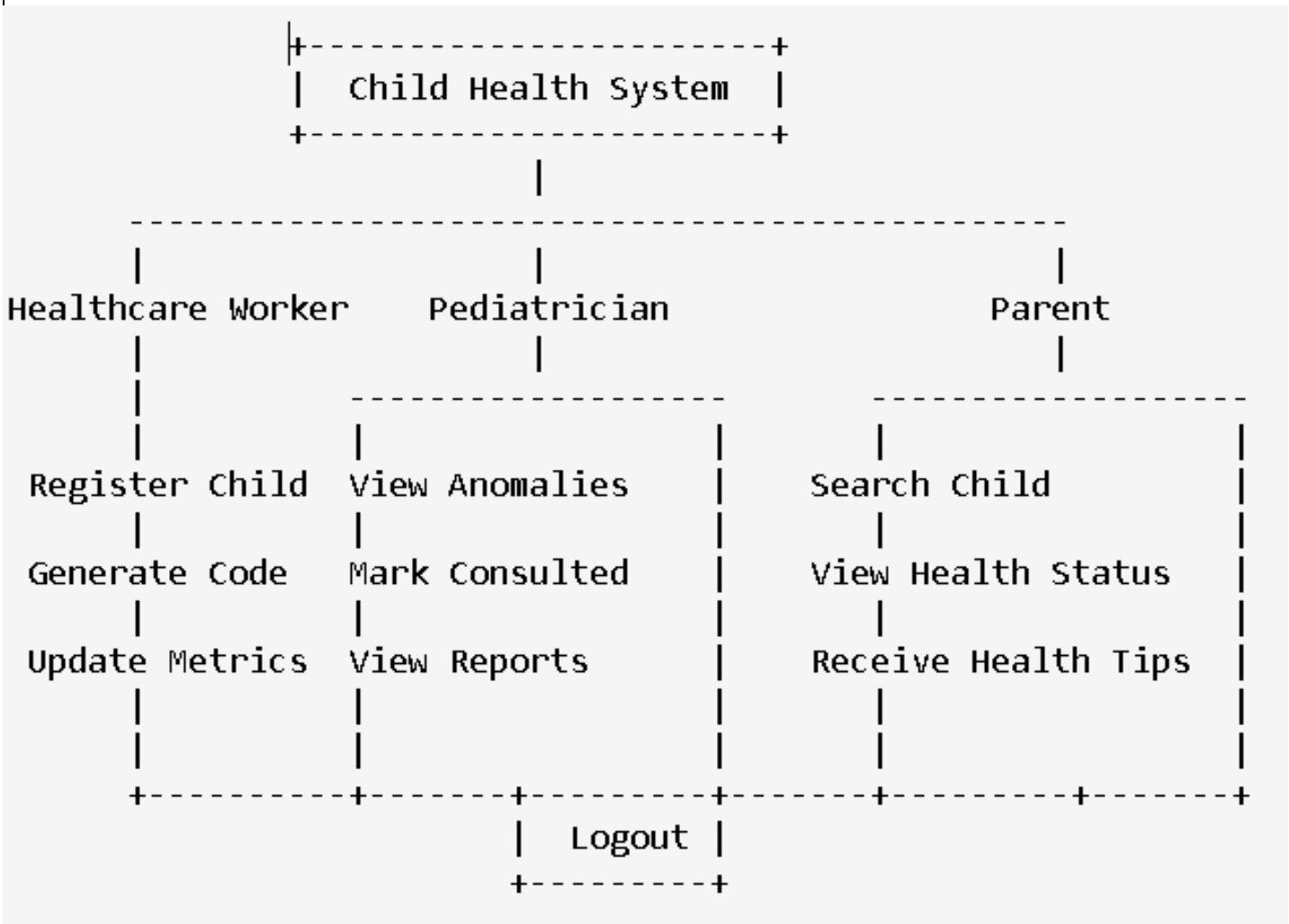
---

7. Health Alert Module (HAM)

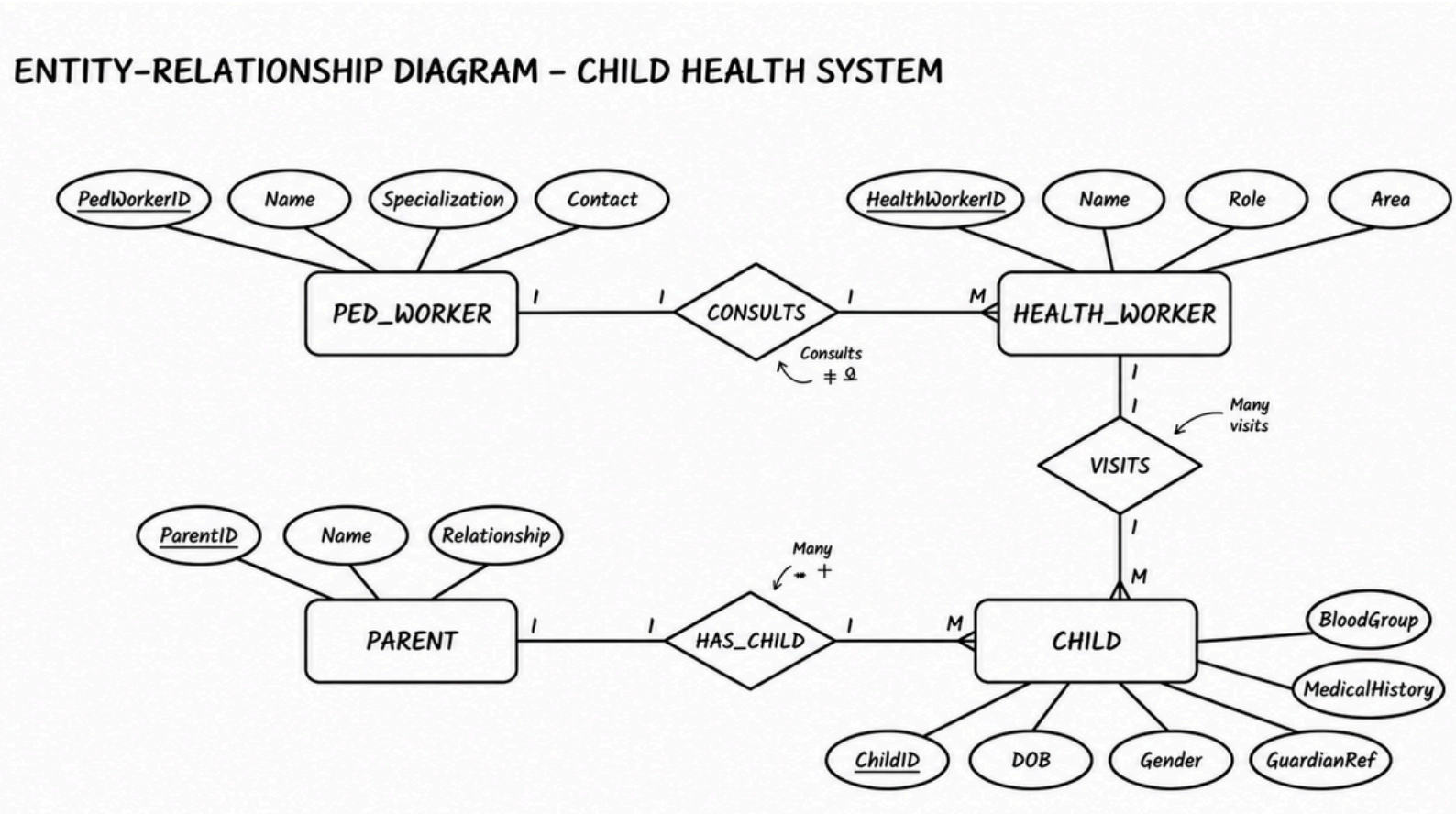
Monitors the recorded data and automatically flags high-priority cases in the "Attention Required" section for pediatricians. This ensures that children falling outside normal growth thresholds receive timely medical attention.

SYSTEM FLOW DIAGRAMS

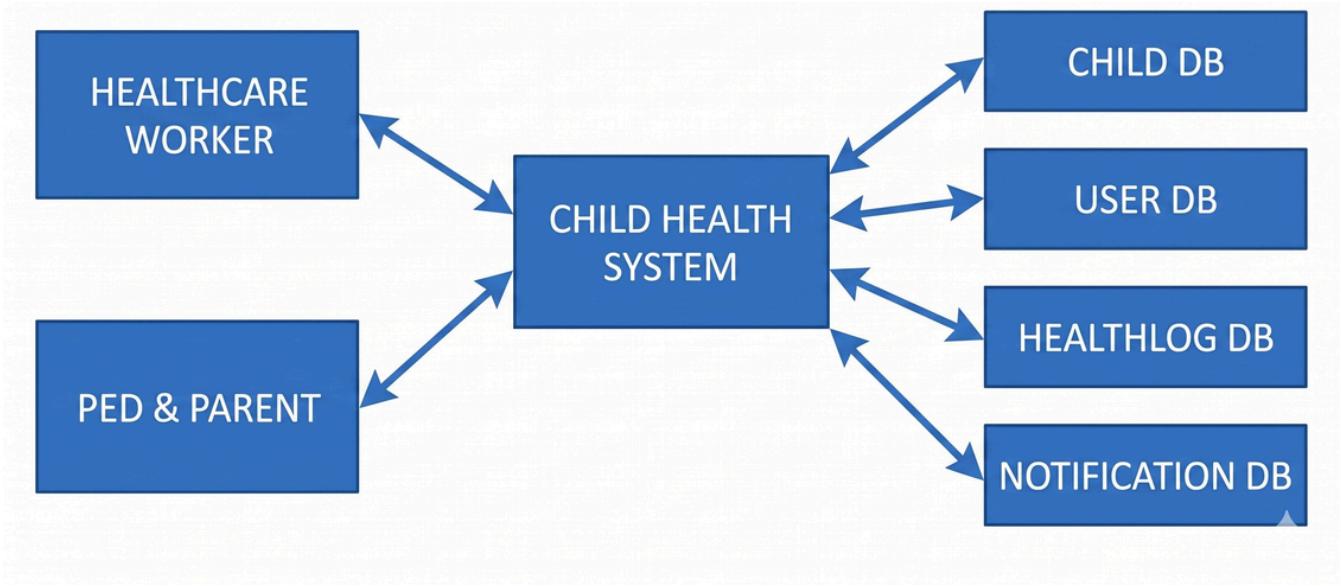
2.1 *USE CASE DIAGRAMS*



2.2 *ENTITY RELATIONSHIP DIAGRAMS*



2.3 DATA FLOW DIAGRAMS



MODULE DESCRIPTION

USER AUTHENTICATION (LOGIN)

Healthcare Workers, Pediatricians, and Parents can log in using their secure credentials. The system verifies the role in the database and grants access to the specific dashboard relevant to that user, ensuring data privacy and security.

CHILD REGISTRATION & GROWTH TRACKING

The Healthcare Worker can register new children by entering details like name, age, height, weight, and blood group. The system automatically generates a unique 6-digit identification code for each child and stores these metrics to monitor growth over time.

HEALTH ANALYTICS & STATUS CALCULATION

The system automatically calculates the child's health status (e.g., Healthy, Obese, or Malnourished) based on the physical metrics provided. It also generates dynamic "Tips for Betterment" tailored to the child's current health condition to assist parents.

SEARCH & HEALTH RECORD RETRIEVAL

Parents and staff can quickly search for a child's health profile using the unique 6-digit code or the child's name. This module retrieves the full health history, including status labels and previous consultation notes, from the Child DB.

PEDIATRICIAN CONSULTATION & ATTENTION DASHBOARD

The Pediatrician accesses a specialized "Attention Required" dashboard that flags children with health anomalies. After reviewing a case, the Pediatrician can update the record to mark it as "Consulted," ensuring that high-priority cases are managed effectively.

NOTIFICATIONS & ALERTS

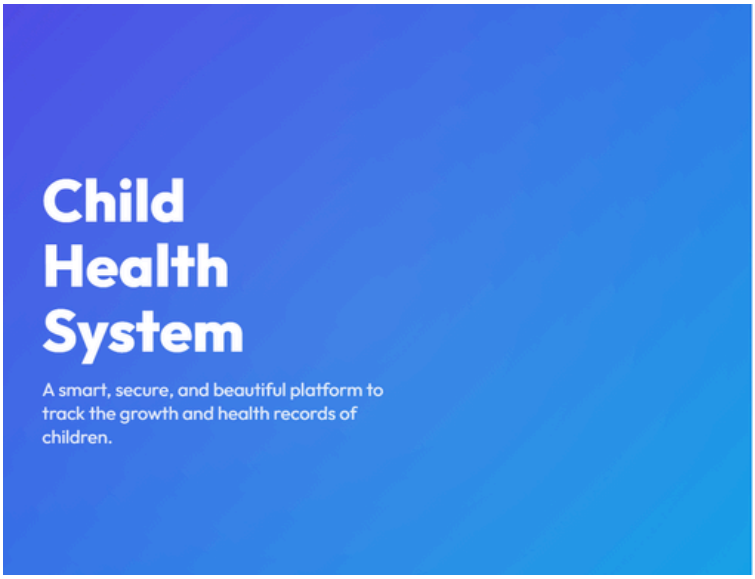
The system continuously monitors data and sends real-time Notifications. These alerts inform users of successful registration, status updates, or high-priority warnings when a child's health status falls into a critical category.

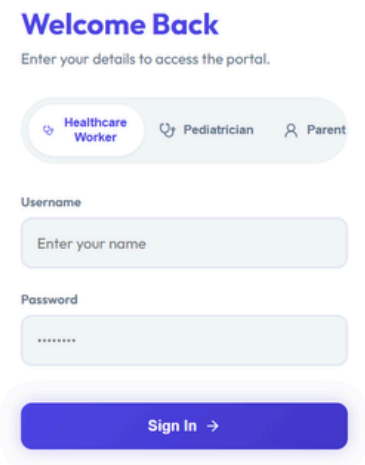
LOGOUT

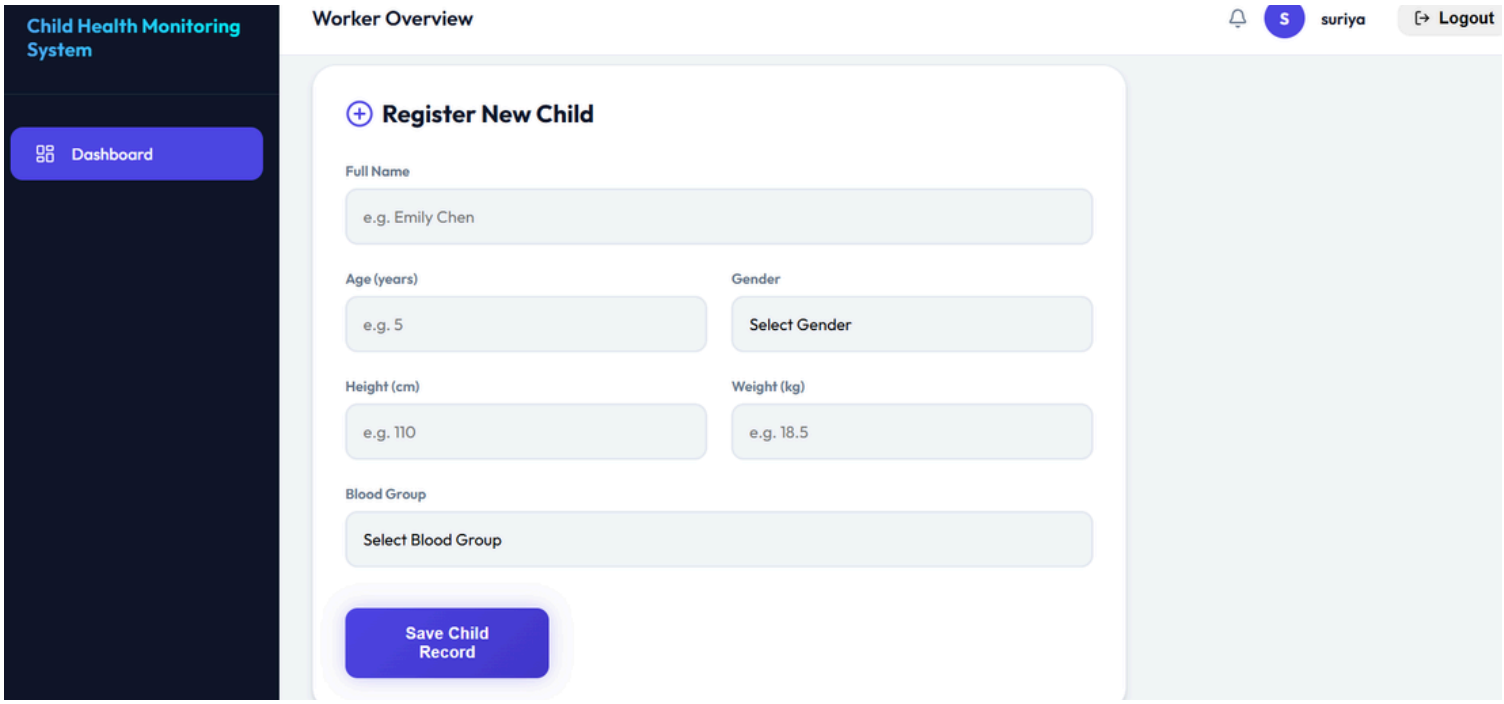
After completing health records management or consultations, users can securely log out to end their session and protect sensitive child health information.

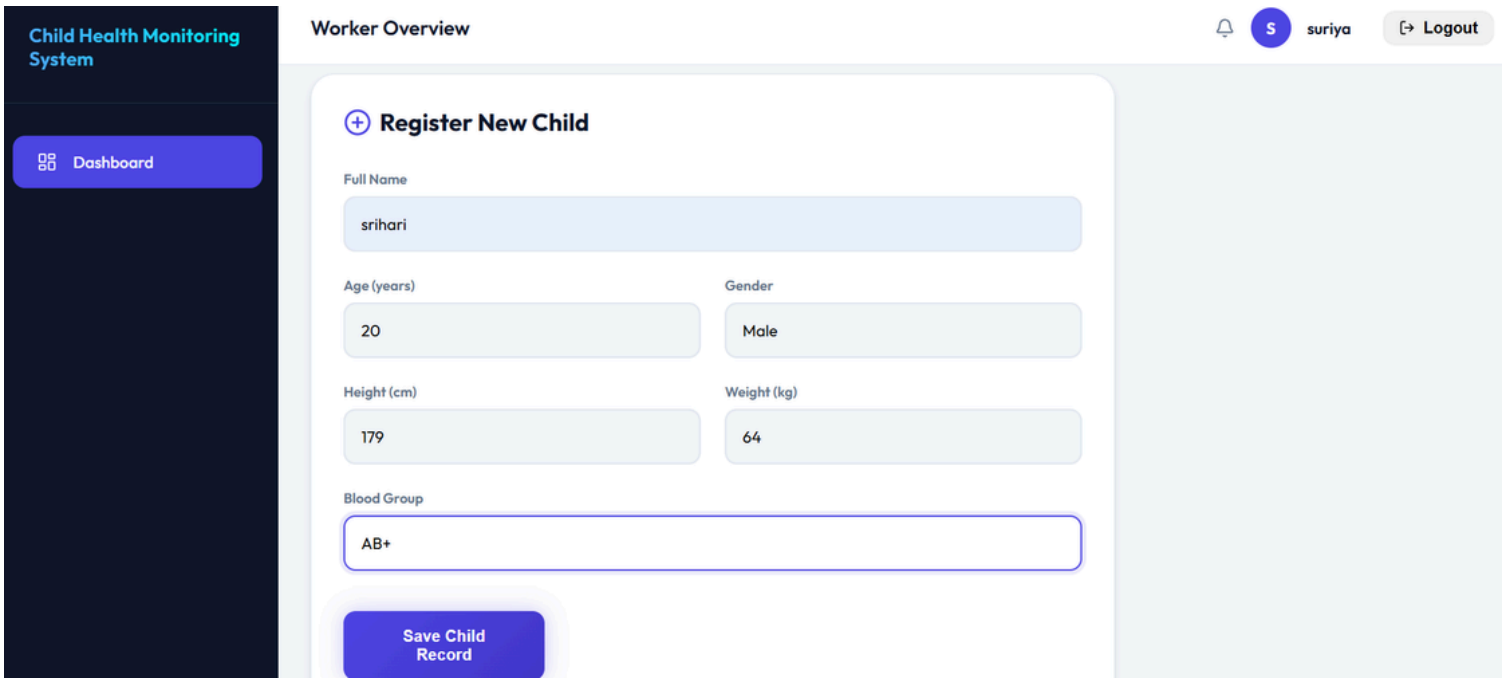
---

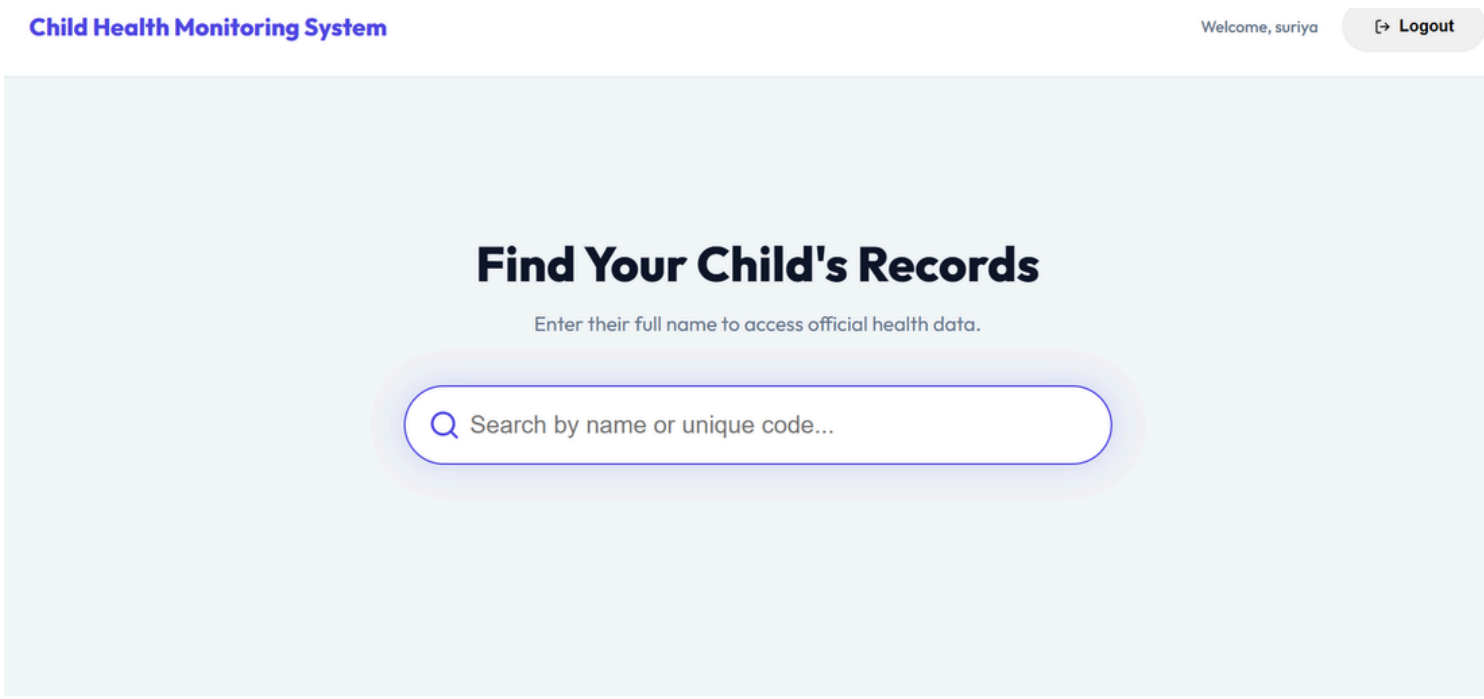
2. IMPLEMENTATION

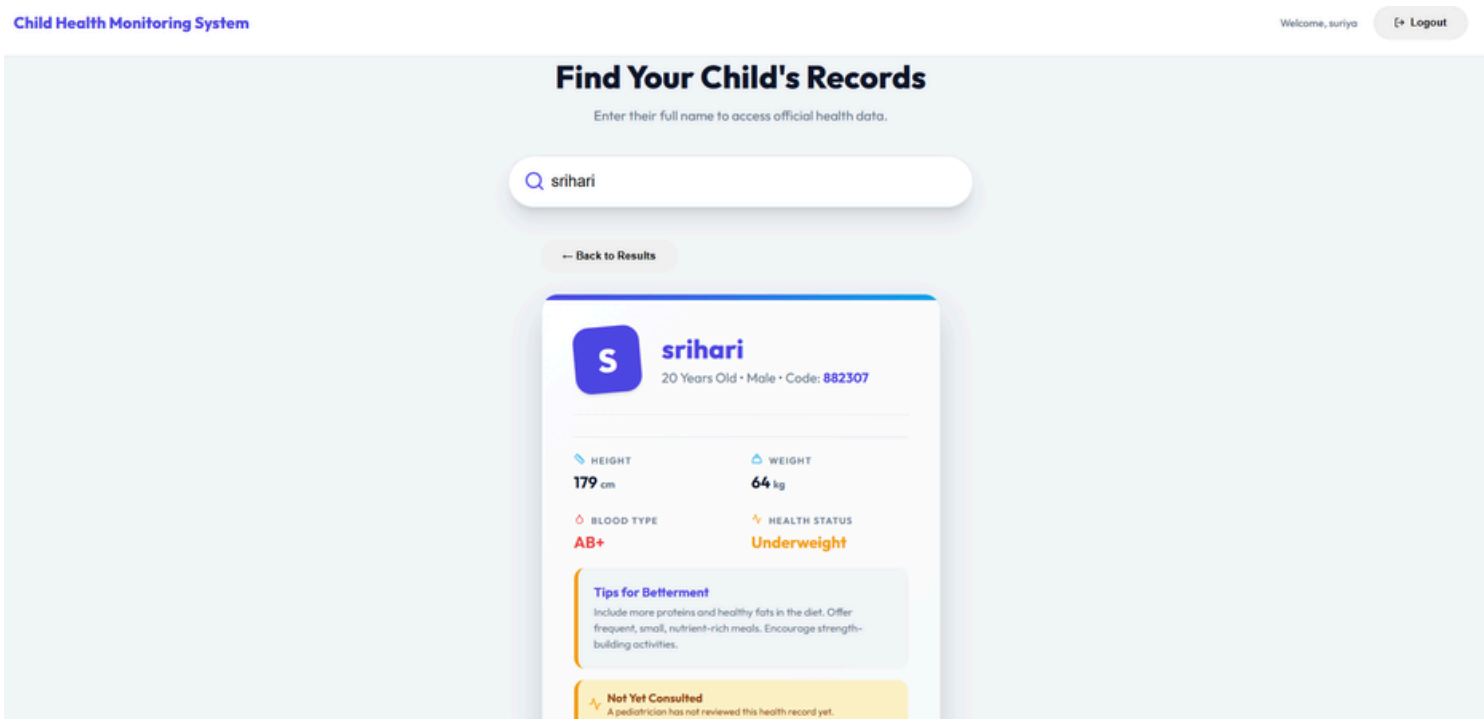


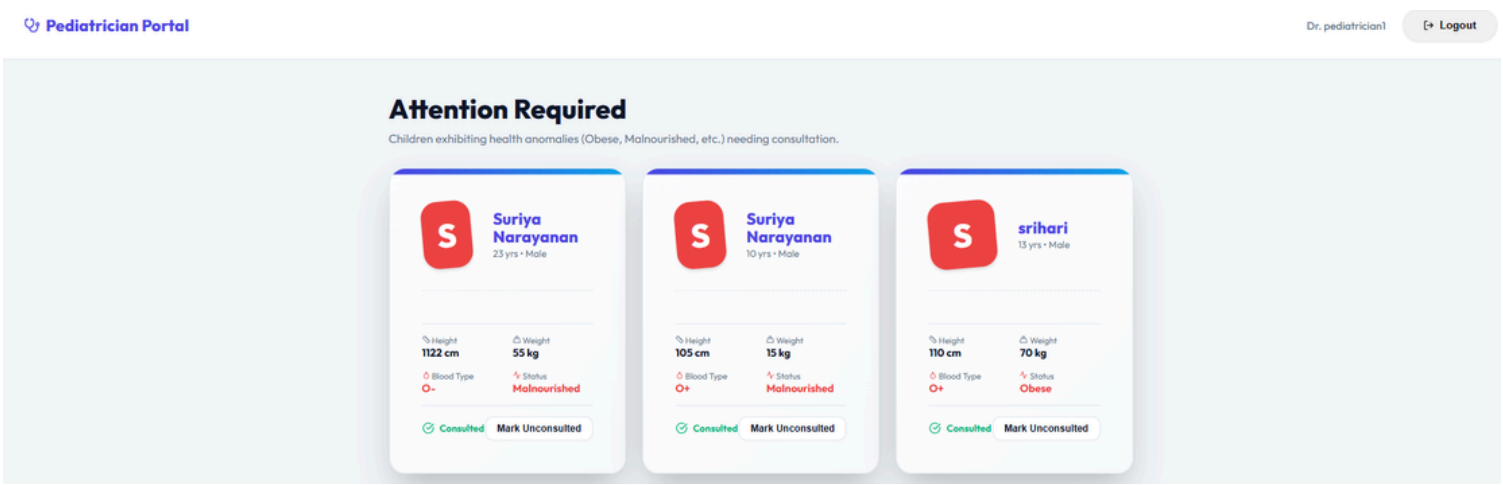



Child Health Monitoring System
Worker Overview
S
suriya
Logout
Dashboard
Register New Child
Full Name
e.g. Emily Chen
Age (years)
e.g. 5
Gender
Select Gender
Height (cm)
e.g. 110
Weight (kg)
e.g. 18.5
Blood Group
Select Blood Group
Save Child Record


Child Health Monitoring System
Worker Overview
S
suriya
Logout
Dashboard
Register New Child
Full Name
srihari
Age (years)
20
Gender
Male
Height (cm)
179
Weight (kg)
64
Blood Group
AB+
Save Child Record


Child Health Monitoring System
Welcome, suriya
Logout
Find Your Child's Records
Enter their full name to access official health data.
Search by name or unique code...


Child Health Monitoring System
Welcome, suriya
Logout
Find Your Child's Records
Enter their full name to access official health data.
srihari
← Back to Results
S
srihari
20 Years Old • Male • Code: 882307
HEIGHT
179 cm
WEIGHT
64 kg
BLOOD TYPE
AB+
HEALTH STATUS
Underweight
Tips for Betterment
Include more proteins and healthy fats in the diet. Offer frequent, small, nutrient-rich meals. Encourage strength-building activities.
Not Yet Consulted
A pediatrician has not reviewed this health record yet.



4.2 DATABASE DESIGN

The data in the Child Health System must be stored and retrieved efficiently using a database to ensure real-time health monitoring and accurate reporting. Designing the database is a critical phase of the system lifecycle, as it establishes how sensitive information—such as child physical metrics, user credentials, and health alerts—is structured and secured.

A database is a collection of interrelated data organized to minimize redundancy and provide fast access for multiple stakeholders. In this system, the primary goal is to ensure that healthcare workers can register children quickly, pediatricians can identify high-priority cases instantly, and parents can access their child's records with high availability.

For this project, a NoSQL database approach is utilized. MongoDB stores information in the form of collections and documents rather than traditional tables and rows. Each record is stored in a JSON-like format (BSON), which offers the flexibility needed to handle dynamic health data, such as varying types of health notifications or evolving growth metrics. This schema-less nature allows the system to remain scalable and adaptable as more complex health analytics are integrated.

4.3 CODE

1. HTML, CSS, JAVASCRIPT
2. MONGODB

1. JAVA LOGIN PAGE CODE

frontend > src > JS App.js > ...

```
1   import React from 'react';
2   import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
3   import Login from './components/Login';
4   import WorkerDashboard from './components/WorkerDashboard';
5   import ParentDashboard from './components/ParentDashboard';
6   import PediatricianDashboard from './components/PediatricianDashboard';
7
8   function App() {
9     return (
10      <BrowserRouter>
11        <Routes>
12          <Route path="/" element={<Login />} />
13          <Route path="/worker" element={<WorkerDashboard />} />
14          <Route path="/parent" element={<ParentDashboard />} />
15          <Route path="/pediatrician" element={<PediatricianDashboard />} />
16          <Route path="*" element={<Navigate to="/" replace />} />
17        </Routes>
18      </BrowserRouter>
19    );
20  }
21
22  export default App;
```

Tab to Jump

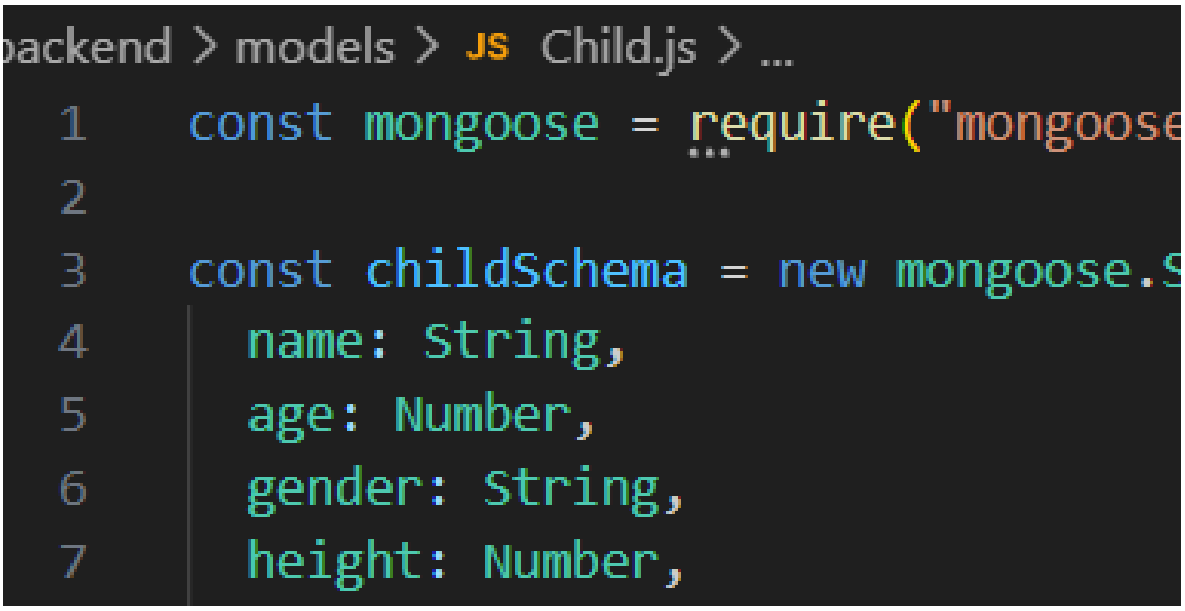
frontend > src > JS index.js > ...

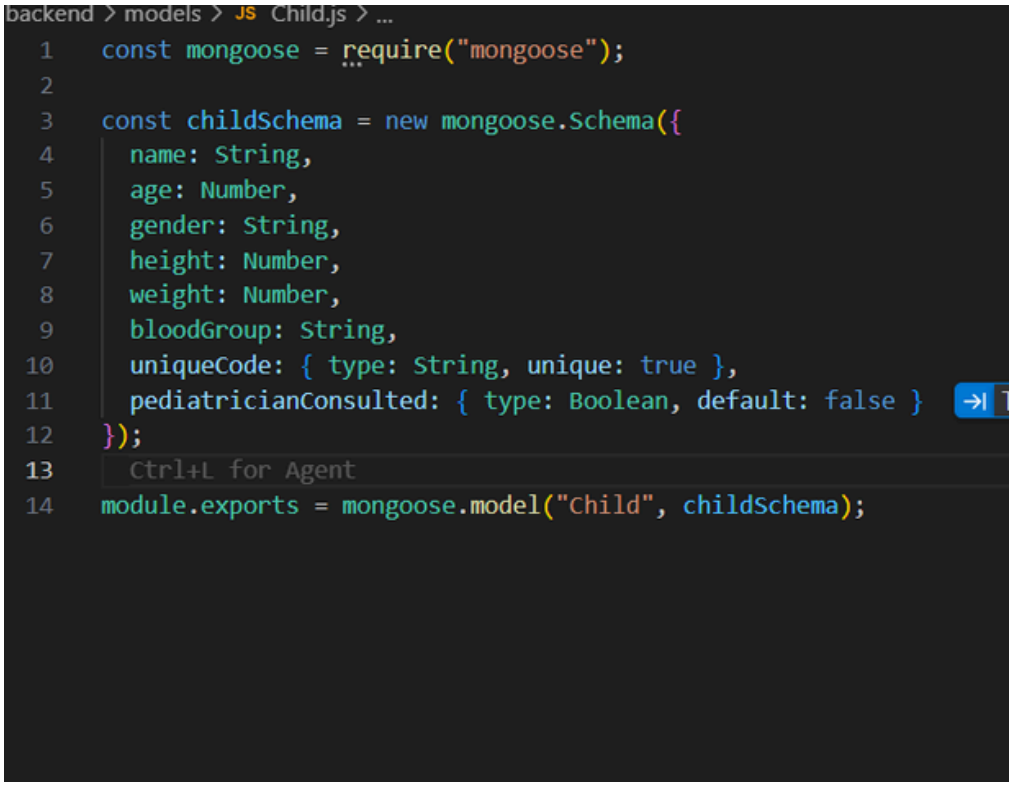
```
1   import React from 'react';
2   import ReactDOM from 'react-dom/client';
3   import './index.css';
4   import App from './App';
5   import reportWebVitals from './reportWebVitals';
6
7   const root = ReactDOM.createRoot(document.getElementById('root'));
8   root.render(
9     <React.StrictMode>
10      <App />
11    </React.StrictMode>
12  );
13
14  // If you want to start measuring performance in your app, pass a function
15  // to log results (for example: reportWebVitals(console.log))
16  // or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals
17  reportWebVitals();
18
```

backend > JS server.js > ...

```
require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const Child = require("./models/Child");
const User = require("./models/User");

const app = express();

app.use(cors());
app.use(express.json());

const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI || "mongodb://127.0.0.1:27017/childhealth";

// MongoDB connection
mongoose.connect(MONGO_URI)
.then(() => {
  console.log("MongoDB Connected");
  seedUsers(); // Seed default users if they don't exist
})
.catch(err => console.log(err));

// Seed Default Users
async function seedUsers() {
  const workerCount = await User.countDocuments({ role: 'worker' });
  if (workerCount === 0) {
    await User.create({ name: "worker1", password: "password123", role: "worker" });
    console.log("Seeded default worker (worker1 / password123)");
  }

  const parentCount = await User.countDocuments({ role: 'parent' });
  if (parentCount === 0) {
```

```
async function seedUsers() {
  const parentCount = await User.countDocuments({ role: 'parent' });
  if (parentCount === 0) {
    await User.create({ name: "parent1", password: "password123", role: "parent" });
    console.log("Seeded default parent (parent1 / password123)");
  }

  const pedCount = await User.countDocuments({ role: 'pediatrician' });
  if (pedCount === 0) {
    await User.create({ name: "pediatrician1", password: "password123", role: "pediatri
    console.log("Seeded default pediatrician (pediatrician1 / password123)");
  }
}

// Test route
app.get("/", (req, res) => {
  res.send("API Working");
});

// Login API
app.post("/login", async (req, res) => {
  const { name, password, role } = req.body;

  if (name && password && role) {
    // Accept any login details entered by the user
    res.json({ success: true, role: role, name: name });
  } else {
    res.status(400).json({ success: false, message: "Please provide all required fields'
  }
});

// Add child (Workers only in UI)
app.post("/child", async (req, res) => {
```


backend > models > JS Child.js > ...
1 const mongoose = require("mongoose
2
3 const childSchema = new mongoose.S
4 name: String,
5 age: Number,
6 gender: String,
7 height: Number,


backend > models > JS Child.js > ...
1 const mongoose = require("mongoose");
2
3 const childSchema = new mongoose.Schema({
4 name: String,
5 age: Number,
6 gender: String,
7 height: Number,
8 weight: Number,
9 bloodGroup: String,
10 uniqueCode: { type: String, unique: true },
11 pediatricianConsulted: { type: Boolean, default: false }
12 });
13 Ctrl+L for Agent
14 module.exports = mongoose.model("Child", childSchema);


backend > models > JS record.js > ...
1 const mongoose = require("mongoose");
2
3 const recordSchema = new mongoose.Schema({
Click to collapse the range.
Tab to Jump
6 weight: Number,
7 date: String
8 });
9
10 module.exports = mongoose.model("Record", recordSchema);

5. CONCLUSION

The Child Health System developed using the MERN stack (MongoDB, Express.js, React, and Node.js) successfully streamlines pediatric health monitoring, including child registration, growth tracking, and health status analytics. By integrating a secure MongoDB database with a dedicated interface for healthcare workers, pediatricians, and parents, the system improves healthcare efficiency, reduces manual record-keeping errors, and supports real-time health data updates.

The system enables effective management of child physical metrics, automated health status calculations (Healthy, Obese, or Malnourished), and critical consultation tracking through the "Attention Required" dashboard. Its flexible NoSQL data structure allows for easy mod on and scalability as medical requirements and the number of registered children growth.

6. REFERENCES

https://www.mongodb.com/docs/manual/

https:/www.w3schools.com/nodejs/

https:/www.w3schools.com/react/

https:/www.geeksforgeeks.org/mern-stack/